Les Graphes : séance 3

Prérequis :

- **Avoir fait les séances précédentes sur les graphes : vocabulaires, matrice d'adjacence et dictionnaire d'adjacence**
- **Écrire la définition d'une classe, coder les attributs et les méthodes d'une classe**
- **Connaître les méthodes des listes et des dictionnaires en Python**

Objectifs :

- **savoir implémenter un graphe orienté ou non sous forme de classe en POO**
- **savoir modifier des méthodes de la classe selon que le graphe soit orienté ou non**

Partie V : création d'une classe Graphe

1er cas : graphe orienté

Nous allons créer une **classe *Graphe_or*** pour implémenter et encapsuler la structure de données graphe orienté.

Cette classe comprend un **unique attribut**, appelé *graph_dict*, qui est un **dictionnaire d'adjacence**, structure de Python vue à l'activité précédente (C36_Graphe$_2$) : **chaque clé est donc un sommet du graphe et chaque valeur est la liste des successeurs du sommet correspondant.**

Pour un rappel sur le dictionnaire et quelques méthodes en Python :https://python.doctor/page-apprendre-dictionnaire-python et/ou http://lptms.u-psud.fr/wiki-cours/index.php/Python:_Dictionnaire

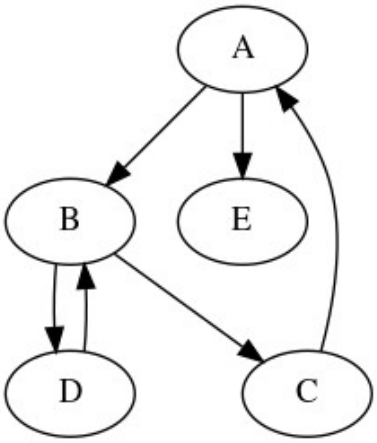
Elle comprend comme premières méthodes :

- le constructeur qui alloue un dictionnaire vide
- « *est_vide* » : retourne *vrai* si le graphe est vide et *faux* sinon
- « *ajouter_sommet* » : ajoute un sommet *s* au graphe s'il n'est pas déjà présent, c'est-à-dire ajoute une clé au dictionnaire avec comme valeur une liste vide
- « *ajouter_arc* » : ajoute un arc qui va d'un sommet s1 à un sommet s2 c'est-à-dire ajoute la valeur s2 à la liste des successeurs du sommet s1. Si les sommets n'existent pas, ils sont également ajoutés.
- « *sommets* » : retourne la liste des sommets du graphe
- « *successeurs* » : retourne la liste des successeurs d'un sommet s si celui-ci existe
- « *arc* » : retourne Vrai si un arc existe allant d'un sommet s1 à un sommet s2

Squelette de la classe *Graphe_or*

```
class Graphe_or:
    # Un graphe comme un dictionnaire d'adjacence

    def __init__(self):
        # crée un dictionnaire pour stocker les listes d'adjacence
        pass

    def est_vide(self):
        # retourne vrai si le graphe est vide
        pass

    def ajouter_sommet(self,s):
        # ajoute un sommet s au graphe s'il n'est pas déjà présent
        pass

    def ajouter_arc(self,s1,s2):
        # ajoute un arc qui va du sommet s1 au sommet s2 (ajoute aussi les sommets s'ils n'existent pas)
        pass

    def successeurs(self,s):
        # retourne la liste des successeurs du sommet s si ce sommet existe
        pass

    def arc(self,s1,s2):
        # retourne vrai s'il existe un arc qui va du sommet s1 au sommet s2
        pass
```

Exercice 1



1. Ouvre et complète le programme *C37_classe_graphe_or.py* en codant la **classe *Graphe_or*** définie précédemment.

2. Teste ce programme et donc toutes les méthodes codées à la question 1. avec le graphe ci-contre.

   Pour cela, tu as besoin de coder la méthode ***get_graphe(self)*** qui retourne *self.graph_dict*. On rappelle que cette méthode qui permet d'accéder à un attribut est appelée **accesseur** (*getter* en anglais).

3. Ajoute les méthodes :
   - ***nb_*sommets*(self)*** qui retourne le nombre de sommets du graphe c'est-à-dire son ordre
   - ***degre(self,s)*** qui retourne le degré d'un sommet s du graphe (c'est-à-dire le nombre d'arcs qui partent de ce sommet) si ce sommet existe
   - ***nb_arcs(self)*** qui retourne le nombre d'arcs du graphe

   Teste ensuite ces trois nouvelles méthodes.

2e cas : graphe non orienté

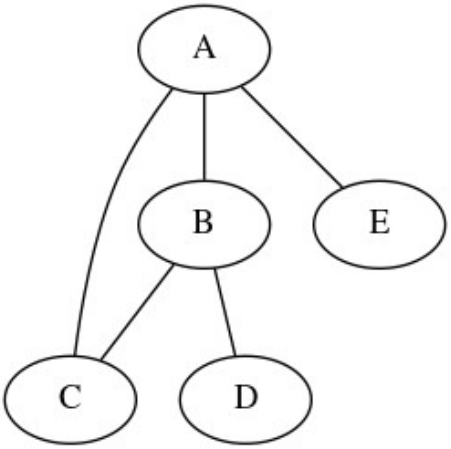
La façon la plus simple de représenter un graphe non orienté consiste à le représenter exactement comme un graphe orienté et d'assurer en permanence l'invariant suivant :

il y a un arc reliant le sommet s1 au sommet s2 si et seulement si il y a un arc reliant le sommet s2 au sommet s1.

Exercice 2 :

1. Reprends le programme de l'exercice 1 en modifiant le nom de la classe : **class *Graphe*,** et enregistre-le sous le nom *C37_classe_graphe.py.*

2. Teste ce programme et donc toutes les méthodes avec le graphe ci-dessous (qui est très proche du précédent). Que remarques-tu ? Que dois-tu modifier ?



2. Effectue les modifications nécessaires dans le programme *C37_classe_graphe.py.*

Pour les plus avancé·e·s

Ajoute deux nouvelles méthodes :

- ***supprimer_arc(self,s1,s2)*** qui supprime l'arc entre s1 et s2 s'il existe.
- ***supprimer_sommet(self,s)*** qui supprime un sommet s et **tous les arcs qui lui sont reliés**.

Exercice 3 :

Nous avons vu précédemment (séance C36 : exercice 6) la fonction ***conversion1*** qui prend en paramètre un graphe orienté *g* sous forme de dictionnaire d'adjacence et renvoie la matrice d'adjacence correspondante.

1. Réécris son code Python dans le cadre ci-dessous en numérotant chaque ligne (normalement de 1 à 11) pour en garder une trace.

2. Transforme cette fonction en une méthode de la **classe *Graphe***. Tu peux la renommer *conversion_mat*.

   Indication 1 : *g* est remplacé par *self* !

   Indication 2 : il faut utiliser les méthodes déjà définies de la classe.

   Indication 3 : à ne lire que si tu es bloqué·e ; clique alors dans le cadre ci-dessous :

   Teste ton programme pour le graphe de l'exercice 2.

   Si l'ordre des sommets est A-B-E-C-D, on obtient la matrice suivante :

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$